

실행결과 화면

```
● (.venv) moon@munjiyeong-ui-MacBookPro python % /Users/moon/workspace/python/.venv/bin/python /Users/moon/workspace/python/skala_datafina
l.py
[pandas] 로딩 시간 =0.367초, 메모리 =741.5MB, shape=(4090836, 20)
[polars] 로딩 시간 =0.127초, 메모리 =549.7MB, shape=(4090836, 20)
-> polars가 pandas 대비 2.90배 빠르고, 메모리는 1.35배 적게 사용

VendorID          int32
tpep_pickup_datetime  datetime64[us]
tpep_dropoff_datetime  datetime64[us]
passenger_count      float64
trip_distance        float64
RatecodeID          float64
store_and_fwd_flag    object
PULocationID        int32
DOLocationID        int32
payment_type         int64
fare_amount          float64
extra                float64
mta_tax              float64
tip_amount           float64
tolls_amount         float64
improvement_surcharge float64
total_amount         float64
congestion_surcharge float64
Airport_fee          float64
cbd_congestion_fee   float64
```

```
VendorID          polars_dtype
tpep_pickup_datetime  Int32
tpep_dropoff_datetime  Datetime(time_unit='us', time_zone=None)
passenger_count      Int64
trip_distance        Float64
RatecodeID          Int64
store_and_fwd_flag    String
PULocationID        Int32
DOLocationID        Int32
payment_type         Int64
fare_amount          Float64
extra                Float64
mta_tax              Float64
tip_amount           Float64
tolls_amount         Float64
improvement_surcharge Float64
total_amount         Float64
congestion_surcharge Float64
Airport_fee          Float64
cbd_congestion_fee   Float64

passenger_count    missing_count  missing_pct
RatecodeID         955371         23.35
store_and_fwd_flag 955371         23.35
congestion_surcharge 955371         23.35
Airport_fee         955371         23.35

완전 중복 행: 0건 (0.000%)
중복 제거 후 shape: (4090836, 20)
처리 후 남은 결측 컬럼: ['passenger_count']
분석 대상 행 수 (중복제거 후): 4,090,836
기간: 2008-12-31 23:05:53 ~ 2026-06-01 00:20:35
fare_amount < 0: 14,231건 (0.348%)
total_amount < 0: 14,877건 (0.364%)
trip_distance == 0: 113,031건 (2.763%)
trip_distance > 100마일: 136건 (0.003%)
하차시각 <= 승차시각: 52,063건 (1.273%)
passenger_count == 0: 12,533건 (0.306%)

trip_distance  fare_amount  tip_amount  total_amount  passenger_count
count  4090836.00  4090836.00  4090836.00  4090836.00  3135465.00
mean    4.95      21.51      2.98      30.49      1.24
std    483.62     19.01      4.02     22.97      0.64
min      0.00    -950.00    -47.10   -951.00      0.00
25%     1.04     10.00      0.00     17.64      1.00
50%     1.87     16.30      2.24     23.94      1.00
75%     3.81     26.80      4.10     34.95      1.00
max    307491.47  5525.99     239.00   5530.74      9.00
```

```

payment_type
1    66.68
0    23.35
2     9.12
4     0.56
3     0.29
Name: pct, dtype: float64
VendorID
2    78.86
1    19.68
7     1.27
6     0.19
Name: pct, dtype: float64
전체 4,090,836건 중 불일치 1,408,196건 (34.42%)
차트 저장 완료: calc_total_vs_total_amount.png
Plotly 인터랙티브 차트 저장 완료: vendor_mismatch_interactive.html
VendorID  전체건수  불일치건수  불일치율(%)
0         1    805221    648351     80.52
1         2   3226222   730820     22.65
2         6     7643     7643    100.00
3         7    51750    21382     41.32
차트 저장 완료: vendor_diff_distribution.png
변수      diff와의 상관계수
0      has_extra      -0.5650
1      is_payment0     0.5634
2  has_congestion     -0.5535
3      is_vendor1     -0.3410
4      is_vendor2     0.2991
5      is_vendor6     0.2772
6      has_cbd_fee     -0.1819
7      is_vendor7      0.0012
차트 저장 완료: diff_correlation_heatmap.png

```

```

검정 우선순위 (상관계수 절댓값 큰 순):
1. has_extra (상관계수=-0.5650)
2. is_payment0 (상관계수=0.5634)
3. has_congestion (상관계수=-0.5535)
4. is_vendor1 (상관계수=-0.3410)
5. is_vendor2 (상관계수=0.2991)
6. is_vendor6 (상관계수=0.2772)
7. has_cbd_fee (상관계수=-0.1819)
8. is_vendor7 (상관계수=0.0012)

```

변수	상관계수	flag=1 평균 diff	flag=0 평균 diff	t통계량	p-value \
0 has_extra	-0.5650	-3.002	4.575	-1024.827	0
1 is_payment0	0.5634	4.631	-2.888	995.172	0
2 has_congestion	-0.5535	-2.938	4.496	-996.943	0
3 is_vendor1	-0.3410	-0.720	3.736	-408.329	0
4 is_vendor2	0.2991	3.560	-0.339	360.300	0
5 is_vendor6	0.2772	26.130	1.551	161.031	0
6 has_cbd_fee	-0.1819	0.807	3.285	-199.505	0
7 is_vendor7	0.0012	1.748	1.684	8.037	9.31e-16

```

Cohen's d
0    -1.4099
1     1.3970
2    -1.3704
3     -0.7276
4     0.6273
5     3.9272
6    -0.3869
7     0.0098
### t-test 결과 해석 (p-value + 효과크기)
- **extra(추가요금) 부과 여부**: p-value=0 -> 통계적으로 유의함 (p < 0.05), 효과크기는 큰 수준 (Cohen's d=-1.410). flag=1 집단 (평균 diff=-3.002)이 flag=0 집단 (평균 diff=4.575) 대비 diff를 낮추는 방향으로 작용함.
- **payment_type=0(미기록) 여부**: p-value=0 -> 통계적으로 유의함 (p < 0.05), 효과크기는 큰 수준 (Cohen's d=1.397). flag=1 집단 (평균 diff=4.631)이 flag=0 집단 (평균 diff=-2.888) 대비 diff를 높이는 방향으로 작용함.
- **congestion_surcharge(혼잡통행료) 부과 여부**: p-value=0 -> 통계적으로 유의함 (p < 0.05), 효과크기는 큰 수준 (Cohen's d=-1.370). flag=1 집단 (평균 diff=-2.938)이 flag=0 집단 (평균 diff=4.496) 대비 diff를 낮추는 방향으로 작용함.
- **VendorID=1(벤더1)**: p-value=0 -> 통계적으로 유의함 (p < 0.05), 효과크기는 중간 수준 (Cohen's d=-0.728). flag=1 집단 (평균 diff=-0.72)이 flag=0 집단 (평균 diff=3.736) 대비 diff를 낮추는 방향으로 작용함.
- **VendorID=2(벤더2)**: p-value=0 -> 통계적으로 유의함 (p < 0.05), 효과크기는 중간 수준 (Cohen's d=0.627). flag=1 집단 (평균 diff=3.56)이 flag=0 집단 (평균 diff=-0.339) 대비 diff를 높이는 방향으로 작용함.
- **VendorID=6(벤더6)**: p-value=0 -> 통계적으로 유의함 (p < 0.05), 효과크기는 큰 수준 (Cohen's d=3.927). flag=1 집단 (평균 diff=26.13)이 flag=0 집단 (평균 diff=1.551) 대비 diff를 높이는 방향으로 작용함.
- **cbd_congestion_fee(CBD혼잡료) 부과 여부**: p-value=0 -> 통계적으로 유의함 (p < 0.05), 효과크기는 작은 수준 (Cohen's d=-0.387). flag=1 집단 (평균 diff=0.807)이 flag=0 집단 (평균 diff=3.285) 대비 diff를 낮추는 방향으로 작용함.
- **VendorID=7(벤더7)**: p-value=9.31e-16 -> 통계적으로 유의함 (p < 0.05), 효과크기는 무시할 수준 수준 (Cohen's d=0.010). flag=1 집단 (평균 diff=1.748)이 flag=0 집단 (평균 diff=1.684) 대비 diff를 높이는 방향으로 작용함.

```

```

표본 크기가 수십만 건에 달해 모든 변수의 p-value가 사실상 0에 가까우므로, 통계적 유의성보다 Cohen's d로 실질적 크기를 판단해야 함. 효과크기가 가장
큰 변수는 **VendorID=6(벤더6)**로, 금액 불일치(diff)의 핵심 원인으로 해석됨.
train shape: (240000, 5) | test shape: (60000, 5)
불일치 비율 - train: 0.344, test: 0.344
/Users/moon/workspace/python/.venv/lib/python3.9/site-packages/sklearn/linear_model/_linear_loss.py:200: RuntimeWarning: divide by zero encountere
d in matmul
  raw_prediction = X @ weights + intercept
/Users/moon/workspace/python/.venv/lib/python3.9/site-packages/sklearn/linear_model/_linear_loss.py:200: RuntimeWarning: overflow encountered in m
atmul
  raw_prediction = X @ weights + intercept
/Users/moon/workspace/python/.venv/lib/python3.9/site-packages/sklearn/linear_model/_linear_loss.py:200: RuntimeWarning: invalid value encountered
in matmul
  raw_prediction = X @ weights + intercept
/Users/moon/workspace/python/.venv/lib/python3.9/site-packages/sklearn/linear_model/_linear_loss.py:330: RuntimeWarning: divide by zero encountere
d in matmul
  grad[:n_features] = X.T @ grad_pointwise + l2_reg_strength * weights
/Users/moon/workspace/python/.venv/lib/python3.9/site-packages/sklearn/linear_model/_linear_loss.py:330: RuntimeWarning: overflow encountered in m
atmul
  grad[:n_features] = X.T @ grad_pointwise + l2_reg_strength * weights
/Users/moon/workspace/python/.venv/lib/python3.9/site-packages/sklearn/linear_model/_linear_loss.py:330: RuntimeWarning: invalid value encountered
in matmul
  grad[:n_features] = X.T @ grad_pointwise + l2_reg_strength * weights
/Users/moon/workspace/python/.venv/lib/python3.9/site-packages/sklearn/linear_model/_linear_loss.py:330: RuntimeWarning: invalid value encountered
in matmul
  grad[:n_features] = X.T @ grad_pointwise + l2_reg_strength * weights
Pipeline 학습 완료
/Users/moon/workspace/python/.venv/lib/python3.9/site-packages/sklearn/utils/extmath.py:203: RuntimeWarning: divide by zero encountered in matmul
  ret = a @ b
/Users/moon/workspace/python/.venv/lib/python3.9/site-packages/sklearn/utils/extmath.py:203: RuntimeWarning: overflow encountered in matmul
  ret = a @ b
/Users/moon/workspace/python/.venv/lib/python3.9/site-packages/sklearn/utils/extmath.py:203: RuntimeWarning: invalid value encountered in matmul
  ret = a @ b
Accuracy = 0.9303
Precision = 0.8466
Recall = 0.9739
F1-score = 0.9058

```

```

Confusion Matrix (행=실제, 열=예측):
[[35713  3643]
 [ 539 20105]]

      precision    recall  f1-score   support

   일 치         0.99         0.91         0.94       39356
  불 일 치         0.85         0.97         0.91       20644

 accuracy         0.92         0.94         0.93       60000
 macro avg         0.92         0.94         0.93       60000
 weighted avg         0.94         0.93         0.93       60000

모델 저장 완료: /Users/moon/workspace/python/mismatch_prediction_pipeline.joblib
report.md 생성 완료

```

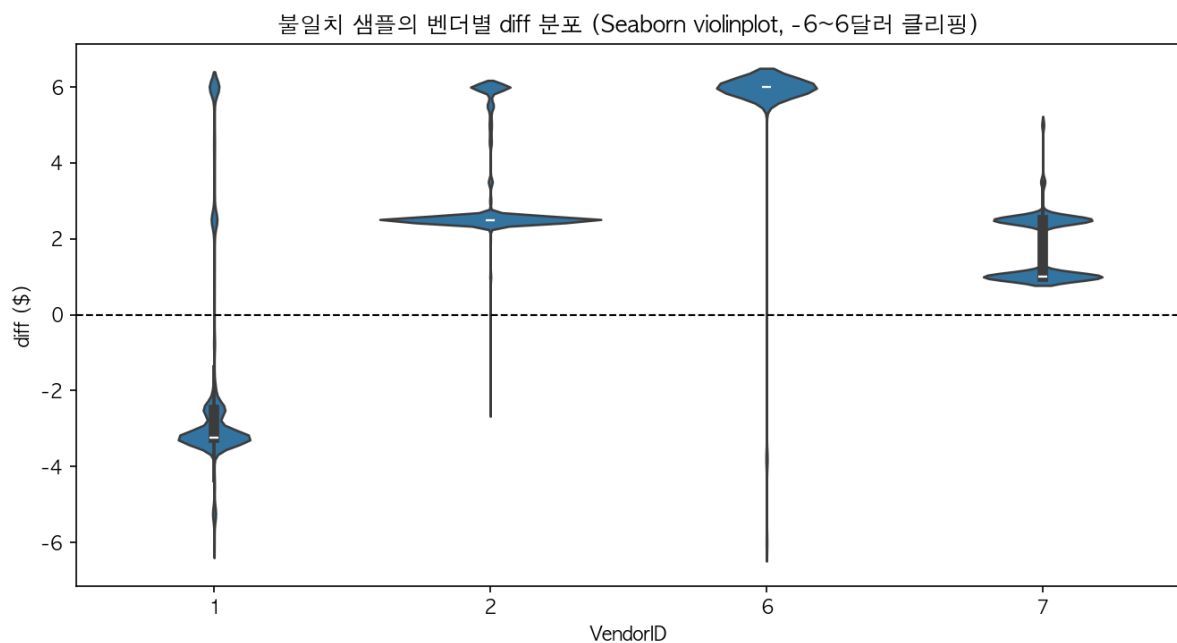


그림1. vendor_diff_distribution

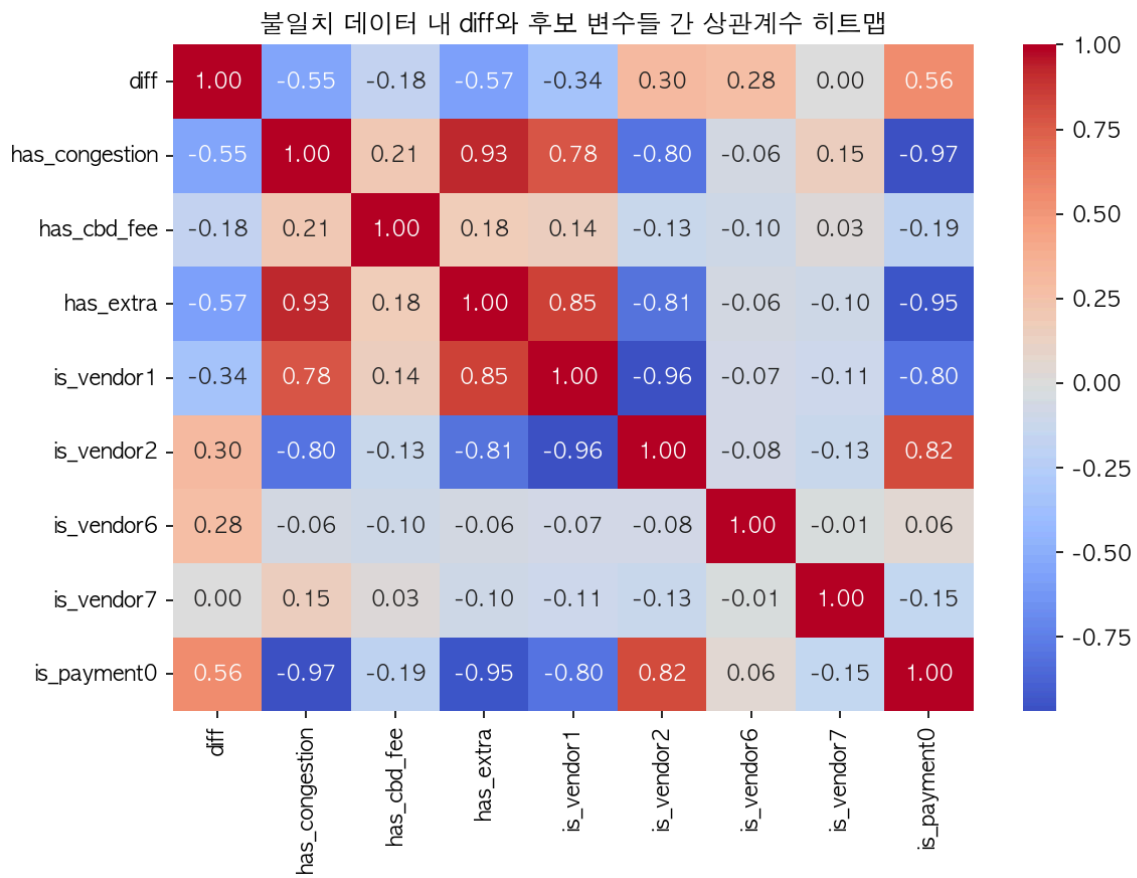


그림2. diff_correlation_heatmap

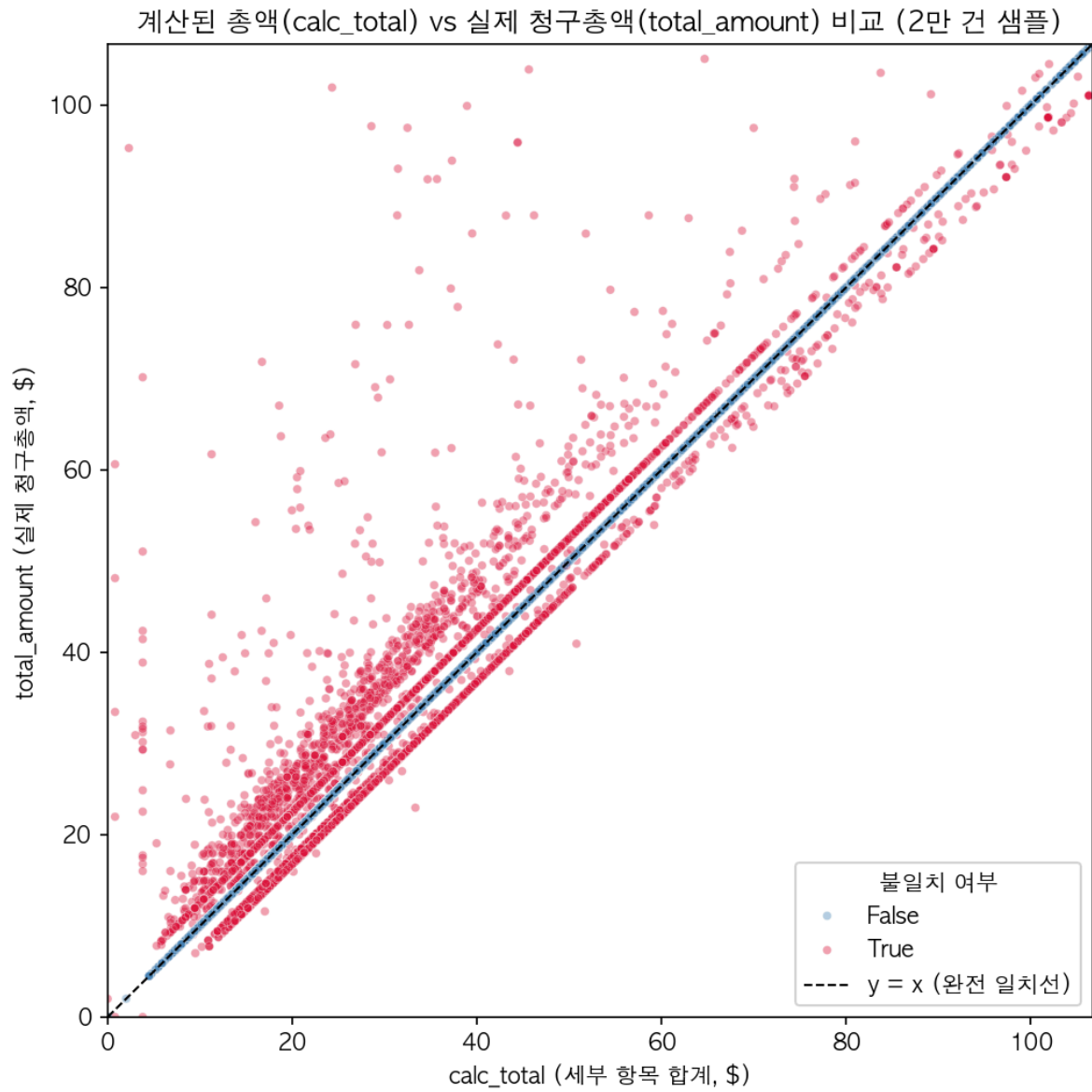


그림3. calc_total_vs_total_amount

팀 종합의견

1. 금액 일치여부 확인의 중요성

- `total_amount` (청구총액)가 이미 존재하는 컬럼이라도, 세부 요금 항목(`fare_amount`, `extra`, `mta_tax`, `tip_amount`, `tolls_amount`, `improvement_surcharge`, `congestion_surcharge`, `Airport_fee`, `cbd_congestion_fee`)의 합과 실제로 일치하는지는 별도로 검증이 필요함
- 이는 회계·감사의 기본 원칙과 동일함: 최종 보고 수치 하나만 신뢰하지 않고 그 구성요소 합과 대조해야 데이터의 신뢰성을 확보할 수 있음
- 본 프로젝트의 데이터셋은 전체 트립의 34.42%에서 이 정합성이 깨져 있었음. 이는 단순한 데이터 품질 문제를 넘어, 실제로는 아래와 같은 경영·법적 리스크와 직결됨
 - 정부에 납부해야 하는 통행료(혼잡통행료 등)가 승객에게 제대로 청구되지 않아 규제 준수 리스크로 이어질 수 있음
 - 기사 정산이 `total_amount` 기준으로 이뤄질 경우, 항목 누락이 기사 또는 회사 어느 쪽의 금전적 손해로도 이어질 수 있음
 - 벤더(결제단말 업체)별로 데이터를 신뢰할 수 있는지 판단하는 근거가 됨
 - 이 값을 그대로 사용하는 모든 후속 분석(매출 예측, 수요 분석 등)의 신뢰성이 걸린 문제임
- 즉 "이미 있는 값이니 그대로 쓴다"가 아니라, 사용 전에 반드시 정합성 검증을 거쳐야 하는 항목이라는 것이 이번 프로젝트의 핵심 전제였음

2. 본 파이프라인이 다루는 범위

이번에 구축한 파이프라인은 다음 두 단계까지를 자동화된 범위로 다룸.

1. **문제 인식 단계:** `diff = total_amount - 세부항목 합` 을 계산해 불일치 여부(`is_mismatch`)를 판별하고, 벤더별 불일치 건수·불일치율을 집계해 "어디에 문제가 있는가"를 시각적으로/수치로 확인
2. **검증 우선순위 도출 단계:** 불일치 데이터만 따로 모아, 후보 변수들(요금 구성요소 유무, 벤더 더미, 결제타입 더미)과 `diff` 간의 상관계수를 계산하고, 상관계수 절댓값이 큰 순서로 "어떤 변수부터 검토해야 하는가"의 우선순위를 자동으로 산출. 이 우선순위를 바탕으로 t-test까지 수행해 통계적 유의성과 효과크기(Cohen's d)를 확인

즉 이 파이프라인은 "불일치가 있다는 것을 발견하고, 그 원인일 가능성이 높은 변수 순서를 통계적으로 정리해주는 단계"까지를 자동화한 것임. 새로운 월별 데이터가 들어와도 같은 코

드로 반복 실행하면 이 지점까지는 자동으로 결과를 얻을 수 있음.

3. 한계점

- 위 파이프라인은 상관계수와 t-test 결과를 통해 "어떤 변수가 불일치와 관련 있어 보인다"는 신호까지만 제공하며, **그 신호가 실제로 어떤 메커니즘 때문에 발생하는지는 규명하지 못함**
- 예를 들어 `has_congestion`, `is_vendor1` 이 모두 diff와 강한 상관관계를 보였지만, 이 표만으로는 "벤더1의 시스템이 애초에 total_amount 계산 시 혼잡통행료를 포함하지 않는다"는 구체적인 원인까지는 알 수 없음
- 상관관계가 섞여 있는 경우(예: `has_congestion=0` 그룹에 벤더1의 계산공식 이슈와 결측 타입 결측 이슈가 동시에 섞여 있는 것)를 이 파이프라인만으로는 구분해내지 못함
- 따라서 이 파이프라인은 "어디부터 파봐야 하는지"를 알려주는 1차 스크리닝 도구이며, 실제 원인 규명에는 사람이 직접 가설을 세우고 검증하는 심층 분석이 추가로 필요함

팀이 추가로 진행한 것

파이프라인이 제시한 우선순위를 바탕으로, 팀에서는 다음과 같이 한 단계 더 깊은 원인 규명을 진행함. 코드 흐름과 각 단계의 실행 결과값을 근거로 삼아, 아래와 같은 의사결정 과정을 그대로 코드에 반영해 재현 가능하도록 작성함(노트북 'STEP 1/2/3' 섹션).

- **STEP 1 (벤더2):** EDA에서 이미 확인한 '특정 5개 컬럼이 동일 비율로 결측된다'는 사실을 diff와 연결하는 가설을 가장 먼저 검증 → 벤더2 불일치의 99.6%가 이 결측 (`payment_type=0`) 그룹과 일치함을 확인 → **기존 결측 이슈의 파생 효과**로 결론
- **STEP 2 (벤더1):** 동일한 결측 가설을 먼저 적용했으나 설명력이 낮아 기각 → diff 값 자체의 분포(이산적 값들)를 확인하는 것으로 방향 전환 → `congestion_surcharge·cbd_congestion_fee` 조합별 diff 중앙값이 정확히 그 금액만큼 부족함을 확인 → 계산식에서 두 항목을 제외했을 때 벤더1만 일치율이 크게 개선 (19.5% → 80.8%)됨을 검증해 **벤더 고유의 계산공식 차이**로 결론(데이터 손실 아님)
- **STEP 3 (벤더7):** 결측 가설, 벤더1식 계산공식 가설을 순서대로 적용했으나 모두 기각 → `extra` 컬럼이 벤더7 전체에서 항상 0으로 고정되어 있다는 사실을 확인 → 불일치 diff 값이 TLC 공식 할증 금액과 정확히 일치함을 근거로 ****특정 컬럼의 구조적 미기록 (실질적 정보 손실)****으로 결론

이 과정을 통해 "34.42%의 불일치"라는 하나의 숫자 뒤에 서로 무관한 세 가지 원인(기존 이슈의 파생효과 / 계산공식 차이 / 컬럼 손실)이 섞여 있었음을 확인했고, 각 원인의 성격이 다르므로 해결 방법도 각각 다르게 적용해야 한다는 결론에 도달함.

개인 discussion

국민혁

이상치 스캔 결과를 print로만 보여주는데, 딕셔너리나 표로 저장해두면 나중에 report.md에서도 그대로 재사용할 수 있어서 중복 작업이 줄 것 같음.

김무겸

결측치 처리 전략(컬럼별로 'Unknown'/0/유지)을 코드에 하드코딩하지 말고, {컬럼명: 처리 방식} 형태의 설정 딕셔너리로 빼두면 다음 달 데이터가 들어왔을 때 컬럼 구성이 바뀌어도 코드를 거의 안 고치고 재사용할 수 있을 것 같음

지금은 완전 중복 행만 제거하는데, 일부 컬럼만 같고 나머지는 다른 준중복 케이스도 있을 수 있으니 주요 식별 컬럼(승차시각+지역+요금 등) 기준 중복도 한번 더 체크하면 더 안전할 것 같음

문지영

STEP 1~3에서 쓴 "결측 확인 → 계산식 확인 → 특정 컬럼 고정값 확인" 패턴을 함수로만 들어두면, 나중에 새 벤더가 추가되거나 새로운 이상치 그룹이 생겨도 같은 체크리스트를 자동으로 돌려볼 수 있어서 더 확장성 있는 코드가 될 것 같음.

이창민

임계값을 지금보다 높이면(예: 0.5달러, 1달러 이상만 불일치로 판단) 전체 34.42%라는 불일치 비율 자체가 줄어들텐데, 벤더별 불일치율·상관계수·t-test 결과·효과크기(Cohen's d)도 함께 달라질 수 있음. 즉 적절한 임계값을 설정해 임계값에 대한 결과의존성을 확인할 필요가 있음.

장연

지금은 노트북 하나에 전부 들어있는데, 데이터 로딩/전처리/검정/ML 부분을 각각 별도 함수나 모듈로 분리해두면 나중에 특정 부분만 재사용하거나 수정하기 훨씬 편할 것 같음.